

Linux :)

study 2편

남궁명수
2026년 6월 16일

제3장. Linux 시스템 운영 도구	3
1. Linux 파일 관리와 작업 자동화	3
2. 네트워크 관련 개념	8
3. 네트워크 보안을 위한 SELinux	14
4. 파이프, 필터, 리디렉션	14
5. 프로세스, 서비스	15
6. 응급 복구	18
6. 커널 모듈과 커널 컴파일	23
제4장. Linux 디스크	26
1. 디스크 추가	26
2. RAID	31

제3장. Linux 시스템 운영 도구

1. Linux 파일 관리와 작업 자동화

1-1. 파일 압축과 파일 묶기

Linux에서는 파일 압축(Compression)과 파일 묶기(Archiving)를 원칙적으로 별개의 프로그램으로 처리한다.

파일 압축은 파일 크기를 줄이는 작업이고, 파일 묶기는 여러 파일과 디렉터리를 하나의 파일로 합치는 작업이다. Windows에서는 ZIP 파일 하나로 묶기와 압축을 동시에 수행하는 경우가 많지만, Linux에서는 보통 먼저 파일을 묶은 뒤 필요하면 다시 압축한다.

대표적인 압축 형식으로는 다음과 같은 것들이 있다.

```
xz
bz2
gz
zip
Z
```

일반적으로 xz와 bz2는 높은 압축률을 제공하며, gzip(gz)은 압축률은 다소 낮지만 처리 속도가 빠르다.

압축 프로그램인 xz는 다음과 같이 사용할 수 있다.

```
xz 파일명
```

압축 해제

```
xz -d 파일명.xz
```

명령을 사용한다.

여러 파일을 하나로 묶을 때는 tar(Tape Archive)를 사용한다.

```
tar cvf my.tar /etc/sysconfig/
```

tar의 주요 옵션은 다음과 같다.

```
c : 묶기(Create)
x : 풀기(Extract)
t : 내용 확인(List)

f : 파일 지정
v : 과정 출력
J : xz 압축 사용
z : gzip 압축 사용
j : bzip2 압축 사용
```

예를 들어 디렉터리를 묶으면서 xz 압축까지 동시에 수행하려면 다음과 같이 사용할 수 있다.

```
tar cJvf backup.tar.xz /etc
```

즉, Linux에서는 tar가 파일을 묶고, xz, gzip, bzip2가 압축을 담당한다고 이해하면 된다.

1-2. 파일 검색 명령어

Linux에서는 다양한 방법으로 파일을 검색할 수 있다.

가장 강력한 명령은 find이다.

```
find [경로] [조건]
```

예를 들어 설정 파일을 찾으려면

```
find /etc -name "*.conf"
```

를 사용할 수 있다.

파일 크기를 기준으로 검색할 수도 있다.

```
find /usr/bin -size +10k -size -100k
```

예를 들어, 임시 파일을 찾아 삭제하려면

```
find /home -name "*.swp" -exec rm {} \;
```

를 사용할 수 있다.

실행 파일의 위치만 확인하려면 which를 사용한다.

```
which ls
```

실행 파일뿐 아니라 메뉴얼 페이지까지 찾으려면 whereis를 사용한다.

```
whereis ls
```

locate는 파일 목록 데이터베이스를 기반으로 매우 빠르게 검색하지만, 실시간 검색은 아니라는 특징이 있다.

```
locate passwd
```

1-3. 시스템 관리 도구

Linux는 네트워크, 방화벽, 서비스 설정을 위한 다양한 관리 도구를 제공한다.

네트워크 설정은 nmtui 또는 nm-connection-editor를 통해 수행할 수 있다.

```
nmtui
```

를 실행하면 텍스트 기반 네트워크 설정 환경이 나타난다.

방화벽 설정은 firewall-config를 사용한다.

```
firewall-config
```

를 실행하면 포트 개방, 서비스 허용 등의 작업을 수행할 수 있다.

서비스 관리 도구로는 ntsysv가 존재하지만, 현재는 대부분 systemctl을 사용한다.

1-4. Cron과 At을 이용한 작업 자동화

Linux 서버에서는 반복적인 작업을 자동으로 수행하는 경우가 많다.

이를 담당하는 서비스가 Cron이다.

Cron은 지정된 시간마다 자동으로 명령을 실행하는 스케줄러이며, 관련 서비스는 crond이다.

```
Systemctl status crond
```

로 상태를 확인할 수 있다.

설정 파일은 다음과 같다.

```
/etc/crontab
```

예를 들어,

```
01**** root run-parts /etc/cron.hourly
```

는 매 시간 1분마다 /etc/cron.hourly 안의 스크립트를 실행하라는 의미이다.

Cron 작업은 다음 디렉터리를 통해서도 관리할 수 있다.

```
/etc/cron.hourly  
/etc/cron.daily  
/etc/cron.weekly  
/etc/cron.monthly
```

예를 들어,

```
cd /etc/cron.monthly  
touch myBackup.sh  
chmod 755 myBackup.sh
```

를 수행하면 매월 자동 실행되는 백업 스크립트를 만들 수 있다.

반면 At은 반복 작업이 아니라 일회성 작업을 예약할 때 사용한다.

예를 들어, 내일 새벽 4시에 한 번만 실행되는 작업을 예약하려면

```
at 4:00 am tomorrow
```

를 사용한다.

예약 목록은

```
at -l
```

로 확인할 수 있고,

```
atrm 작업번호
```

로 삭제할 수 있다.

즉, Cron은 반복 작업, At은 일회성 작업을 예약하는 도구라고 이해하면 된다.

1-5. date

추가로 서버 시간을 수정할 때에는 date 명령어를 사용한다.

예를 들어

```
date 011503002035
```

date 01[월] 15[일] 03[시] 00[분] 2035[년]으로 이해하면 된다.

만약 corn을 설정했다면

```
systemctl restart crond
```

명령어를 입력한다.

원래의 시간대로 되돌리고 싶다면

```
rdate -s time.bora.net
```

을 입력한다.

2. 네트워크 관련 개념

2-1. 네트워크 개념

TCP/IP

- 컴퓨터끼리 네트워크 상으로 의사소통을 하는 “프로토콜” 중 가장 널리 사용되는 프로토콜의 한 종류

호스트와 도메인

- 호스트 이름은 각각의 컴퓨터에 지정된 이름
- 도메인 이름(또는 도메인 주소)는 hanbit.co.kr과 같은 형식

IP 주소

- 각 컴퓨터의 랜카드에 부여되는 중복되지 않는 유일한 주소
- 4바이트로 이루어져 있으며, 각자리는 0~255까지의 숫자
- 예) Server의 IP 주소는 192.168.111.100

네트워크 주소

- 같은 네트워크에 속해 있는 공통된 주소(예: 192.168.111.0)

브로드캐스트 주소

- 내부 네트워크의 모든 컴퓨터가 듣게 되는 주소
- 현재 주소의 제일 끝자리를 255로 바꾼 주소(C클래스)

게이트웨이, 라우터

- 라우터 = 게이트웨이
- 네트워크 간에 데이터를 전송하는 컴퓨터 또는 장비
- VMware의 게이트웨이 주소는 192.168.11.2로 고정

넷마스크, 클래스

- 넷마스크: 네트워크의 규모를 결정(예: 255.255.255.0 - C클래스)

DNS(Domain Name System) 서버(=네임서버) 주소

- URL을 해당 컴퓨터의 IP 주소로 변환해 주는 서버
- 설정 파일은 /etc/resolv.conf
- VMware를 사용하면 VMware가 192.168.111.2번을 게이트웨이 및 DNS 서버로, 192.168.111.254를 DHCP 서버로 설정함

리눅스에서의 네트워크 장치 이름

- Rocky Linux 10은 랜카드를 ens160(또는 enp2s0)으로 인식함
- 이전 버전에서는 eth0, eth1, ens32, ens33 등으로 인식했음
- VMware에서 Rocky Linux를 설치할 경우에는 VMware 버전에 따라서 완전히 다른 이름으로 인식할 수도 있음

2-2. 네트워크 관련 명령어

nmtui

- 네트워크와 관련된 대부분의 작업을 이 명령어에서 수행
 - 자동 IP 주소 또는 고정 IP 주소 사용 결정
 - IP 주소, 서브넷 마스크, 게이트웨이 정보 입력
 - DNS 정보 입력
 - 네트워크 카드 드라이버 설정
 - 네트워크 장치(ens160)의 설정
- 텍스트 기반으로 작동함

systemctl <start/stop/restart/status> NetworkManager

- 네트워크의 설정을 변경한 후에, 변경된 내용을 시스템에 적용시키는 명령어

ifconfig <장치이름> up 및 ifup <장치이름>

- 네트워크 장치를 On 시키는 명령어

ifconfig <장치이름> down 및 ifdown <장치이름>

- 네트워크 장치를 Off 시키는 명령어

ifconfig <장치이름>

- 장치의 IP주소 설정 정보를 출력

nslookup

- DNS 서버의 작동을 테스트 하는 명령어

ping <IP 주소 또는 URL>

- 해당 컴퓨터가 네트워크 상에서 응답하는지를 테스트하는 간편한 명령어
- ping -s 5 www.google.com

2-3. 네트워크 설정과 관련된 주요 파일

/etc/NetworkManager/system-connection/장치이름.nmconnection

- 장치(ens160 또는 enp2s0)에 설정된 네트워크 정보가 모두 들어 있는 파일

/etc/resolv.conf

- DNS 서버의 정보 및 호스트 이름이 들어 있는 파일

/etc/hosts

- 현 컴퓨터의 호스트 이름 및 FQDN이 들어 있는 파일

위 파일을 직접 편집하면 nmtui를 사용하지 않아도 됨

2-4. /etc/resolv.conf

Linux에서 네트워크를 사용하기 위해서는 IP 주소뿐만 아니라 DNS(Domain Name System) 설정도 필요하다. DNS는 사람이 이해하기 쉬운 도메인 이름을 실제 IP 주소로 변환해 주는 역할을 수행한다. 이러한 DNS 서버 정보는 일반적으로 /etc/resolv.conf 파일에 저장된다.

예를 들어, /etc/resolv.conf 파일을 확인하면 다음과 같은 내용을 볼 수 있다.

```
cat /etc/resolv.conf
```

```
nameserver 8.8.8.8
nameserver 1.1.1.1
```

여기서 nameserver는 DNS 서버의 주소를 의미한다.

사용자 웹 브라우저에서 www.google.com과 같은 도메인을 입력하면 시스템은 /etc/resolv.conf에 등록된 DNS 서버에 질의하여 해당 도메인의 IP 주소를 조회한다.

```
www.google.com
  ↓
DNS 서버 조회
  ↓
IP 주소 획득
  ↓
해당 서버 접속
```

다만 최근 Linux 시스템에서는 NetworkManager나 systemd-resolved가 DNS 설정을 자동으로 관리하는 경우가 많다. 따라서 사용자가 /etc/resolv.conf 파일을 직접 수정하더라도 네트워크 서비스를 재시작하면 기존 설정이 사라질 수 있다.

즉,

```
/etc/resolv.conf 직접 수정
  ↓
네트워크 서비스 재시작
  ↓
자동 생성
  ↓
설정 변경 내용 사라짐
```

2-5. nslookup을 이용한 DNS 문제 확인

인터넷이 되지 않을 때는 먼저 문제가 네트워크 자체인지, DNS인지 확인해야 한다.

이때 사용할 수 있는 대표적인 명령어가 nslookup이다.

nslookup은 DNS 서버에 직접 질의하여 도메인 이름을 IP 주소로 변환할 수 있는지 확인하는 도구이다.

예를 들어, 다음과 같이 실행할 수 있다.

```
nslookup
```

실행 후

```
www.google.com
```

을 입력한다.

정상적인 경우에는 다음과 같이 IP주소가 반환된다.

```
www.google.com
```

↓

```
DNS 서버 조회
```

↓

```
142.x.x.x
```

만약 IP 주소를 반환하지 못하거나 오류가 발생한다면 DNS 서버에 문제가 있을 가능성이 있다.

2-6. 특정 DNS 서버를 이용한 테스트

현재 설정된 DNS 서버가 정상 동작하지 않는다고 의심될 경우, Google Public DNS와 같은 외부 DNS 서버를 직접 지정하여 테스트할 수 있다.

예를 들어, nslookup 실행 후

```
server 8.8.8.8
```

을 입력한다.

그러면 DNS 질의 대상 서버가 Google DNS로 변경된다.

현재 DNS 서버



문제 발생



server 8.8.8.8



Google DNS 사용

이후 다시

www.google.com

을 조회했을 때 정상적으로 IP 주소가 반환된다면, 기존 DNS 서버에 문제가 있었다고 판단할 수 있다.

2-7. 정리

/etc/resolv.conf는 Linux에서 DNS 서버 정보를 저장하는 중요한 설정 파일이다.

사용자가 도메인 이름을 입력하면 시스템은 이 파일에 등록된 DNS 서버를 이용하여 IP 주소를 조회한다. 다만 NetworkManager나 systemd-resolved가 DNS 설정을 자동으로 관리하는 환경에서는 수동으로 수정한 내용이 사라질 수 있다.

또한 인터넷 연결에 문제가 발생했을 때는 nslookup 명령어를 사용하여 DNS 서버가 정상적으로 동작하는지 확인할 수 있으며, server 8.8.8.8과 같이 특정 DNS 서버를 지정하여 문제의 원인을 진단할 수 있다.

3. 네트워크 보안을 위한 SELinux

SELinux(Security Enhanced Linux)는 보안에 취약한 리눅스를 보호하기 위해 탄생
SELinux의 활성화 여부는 `sestatus` 또는 `getenforce` 명령을 실행해 확인할 수 있음

비활성: `grubby --update-kernel ALL --args selinux = 0`
활성: `grubby --update-kernel ALL --remove-args selinux`

`System-config-selinux` 명령의 [현재 강제 모드]의 작동 방식

- 강제(enforcing)는 시스템 보안에 영향을 미치는 기능이 감지되면 아예 그 기능이 작동되지 않도록 시스템에서 막아줌
- 허용(Permissive)은 시스템 보안에 영향을 미치는 기능이 감지되면 허용은 되지만 그 내용이 로그에 남음
- 비활성화(Disabled)은 SELinux를 사용하지 않음

4. 파이프, 필터, 리디렉션

파이프(pipe)

- 두 개의 프로그램을 연결해 주는 연결통로의 의미
- | 문자를 사용
- 예) `#ls -l /etc | more`

필터(filter)

- 필요한 것만 걸러주는 명령어
- `grep`, `tail`, `wc`, `sort`, `grep`, `awk`, `sed` 등
- 주로 파이프와 같이 사용
- 예) `#ps -ef | grep bash`

리디렉션(redirection)

- 표준 입출력의 방향을 바꿔줌
- 예) `ls -l >list.txt`
- `sort <list.txt > out.txt`

5. 프로세스, 서비스

5-1. 프로세스(process)

프로세스의 정의

- 하드디스크에 저장된 실행 코드(프로그램)가, 메모리에 로딩되어 활성화 된 것, '데몬'으로도 부름

포그라운드 프로세스(Foreground Process)

- 실행하면 화면에 나타나서 사용자와 상호작용을 하는 프로세스
- 대부분의 응용프로그램

백그라운드 프로세스(Background Process)

- 실행은 되었지만, 화면에는 나타나지 않고 실행되는 프로세스
- 백신 프로그램, 서버 데몬 등

프로세스 번호

- 각각의 프로세스에 할당된 고유 번호

작업 번호

- 현재 실행되고 있는 백그라운드 프로세스의 순차번호

부모 프로세스와 자식 프로세스

- 모든 프로세스는 부모 프로세스를 가지고 있음
- 부모 프로세스를 kill 하면, 자식 프로세스도 자동으로 kill 됨

ps

- 현재 프로세스의 상태를 확인하는 명령어
- ps -ef | grep <프로세스 이름>

kill

- 프로세스를 강제로 종료하는 명령어
- kill -9<프로세스 번호>

pstree

- 부모 프로세스와 자식 프로세스의 관계를 트리 형태로 보여줌

예시로

yes > /dev/null	
ctrl + j	: 멈춤(잠깐 일시정지 개념)
bg	: 백그라운드로 보냄
jobs	: 백그라운드에서 작업 중인 작업들
Fg	: 다시 앞으로 가져와서 수행

백그라운드에서 gnome-text-editor에서 사용하고 싶다면 바로 뒤에다가 &를 붙이면 된다.

gnome-text-editor &

5-2. 서비스와 소켓

서비스

- 시스템과 독자적으로 구동되어 제공하는 프로세스를 말한다. 예로 웹 서버(httpd), DB(mysql), FTP 서버(vsftpd) 등이 있다.
- 실행 및 종료는 대개 'systemctl start/stop/restart 서비스 이름'으로 사용된다.
- 서비스의 실행 스크립트 파일은 /usr/lib/systemd/system/ 디렉터리에 '서비스이름.service'라는 이름으로 확인할 수 있다. 예를 들어, Cron 서비스는 crond.service라는 이름의 파일로 존재한다.

소켓

- 서비스는 항상 가동되지만, 소켓은 외부에서 특정 서비스를 요청할 경우에 systemd가 구동시킨다. 그리고 요청이 끝나면 소켓도 종료된다.
- 그래서 소켓으로 설정된 서비스를 요청할 때는 처음 연결되는 시간이 앞에서 설명한 서비스에 비교했을 때 약간 더 걸릴 수 있다. 왜냐하면 systemd가 서비스를 새로 구동하는 데 시간이 소요되기 때문이다. 이와 같은 소켓의 대표적인 예로 텔넷 서버를 들 수 있다.
- 소켓과 관련된 스크립트 파일은 /usr/lib/systemd/system/ 디렉터리에 '소켓이름.socket'이라는 이름으로 존재한다.

6. 응급 복구

6-1. Root 비밀번호 분실 및 복구

Linux에서는 root 비밀번호를 잊어버렸더라도 시스템에 물리적으로 접근할 수 있다면 부팅 과정에서 복구가 가능하다. 이 방법은 운영체제가 완전히 실행되기 전에 커널 부팅 옵션을 수정하여 응급 복구 모드로 진입하는 방식이다.

시스템 부팅 시 GRUB 메뉴 환경에서 Shift + e를 눌러 부팅 항목을 편집한다. 그러면 다음과 같은 커널 부팅 옵션을 확인할 수 있다.

```
Linux ($root)/boot/vmlinuz-6.12.0-55.12.1.el10_0.aarch64 root=UUID=da0dbdbe-  
c1e7-4b52-9e88-ae7582d60ca7 ro  
crashkernel=2G-4G:256M,4G:256M,4G-64G:320M, 64G-:576M rhgb quiet selinux=0
```

커서로 해당 줄의 맨 끝까지 이동한 뒤 다음 내용을 추가한다.

```
init=/bin/sh
```

이 옵션은 systemd나 일반 로그인 과정을 거치지 않고 /bin/sh 셸을 바로 실행하도록 지정하는 것이다. 수정 후 Ctrl+x를 누르면 해당 설정으로 부팅이 진행된다.

부팅이 완료되면 일반 로그인 화면이 나타나지 않고 root 권한의 셸이 실행된다.

즉, 이미 최고 관리자 권한을 가진 상태이므로 비밀번호를 변경할 수 있다.

따라서 다음 명령으로 root 비밀번호를 변경하려고 한다.

```
passwd
```

하지만 이 상태에서 비밀번호를 변경하려고 하면 정상적으로 저장되지 않는 문제가 발생한다. 그 이유는 현재 루트 파일 시스템(/)이 읽기 전용(Read only)로 마운트되어 있기 때문이다.

현재 마운트 상태는 다음 명령으로 확인할 수 있다.

```
mount
```

출력 결과를 보면 다음과 같은 형태를 확인할 수 있다.

```
/dev/sda2 on / type xfs (ro, ...)
```

여기서 /는 Linux의 최상위 디렉터리이며, ro(Read Only) 옵션이 적용되어 있다는 의미이다. 읽기 전용 상태에서는 파일 수정이 불가능하므로 /etc/shadow 파일에 저장되는 비밀번호 정보 역시 변경할 수 없다.

따라서 먼저 루트 파일 시스템을 읽기/쓰기(Read Write) 모드로 다시 마운트해야 한다.

```
mount -o remount,rw /
```

이 명령은 현재 마운트된 루트 파일 시스템을 다시 마운트하면서 rw(Read Write) 옵션을 적용하는 것이다.

이후 다시 마운트 상태를 확인하면 다음과 같이 변경된 것을 볼 수 있다.

```
/dev/sda2 on / type xfs (rw, ...)
```

이제 시스템 파일을 수정할 수 있으므로 root 비밀번호를 정상적으로 변경할 수 있다.

```
passwd
```

새 비밀번호를 두 번 입력하면 /etc/shadow 파일에 암호 정보가 저장된다.

비밀번호 변경이 완료되면 시스템을 재부팅한다.

```
exec /sbin/init 또는 reboot -f
```

재부팅 후에는 새로 설정한 root 비밀번호로 로그인할 수 있다.

정리하면, Root 비밀번호 복구는 GRUB에서 init=/bin/sh 옵션을 추가하여 단일 사용자 모드로 진입한 뒤, 읽기 전용으로 마운트된 루트 파일 시스템을 mount -o remount, rw / 명령으로 읽기/쓰기 모드로 변경하고, passwd 명령을 이용하여 새로운 비밀번호를 설정하는 과정으로 이루어진다. 이 방법은 Linux 시스템 관리자가 반드시 알아두어야 하는 대표적인 긴급 복구 절차 중 하나이다.

6-2. GRUB2 부트로더와 부팅 설정 관리

Linux 시스템이 부팅될 때 가장 먼저 운영체제를 찾아 실행하는 프로그램을 부트로더(Boot Loader)라고 한다. Rocky Linux를 비롯한 대부분의 현대 Linux 배포판에서는 GRUB2(GRand Unified Bootloader 2)를 사용한다.

GRUB2는 단순히 운영체제를 실행하는 역할만 하는 것이 아니라, 사용자가 부팅 과정에서 커널 옵션을 수정하거나 여러 운영체제 중 하나를 선택하여 부팅할 수 있도록 지원한다. 따라서 시스템 장애가 발생했을 때 복구 작업을 수행하거나, Root 비밀번호를 초기화하는 과정에서도 GRUB2를 활용하게 된다.

GRUB 부트로더의 특징

GRUB의 가장 큰 특징은 사용자가 특정 부팅 정보를 직접 수정할 수 있다는 점이다. 운영체제가 정상적으로 부팅되지 않더라도 커널 옵션이나 부팅 경로를 수정하여 부팅을 시도할 수 있다. 실제로 Root 비밀번호 복구 과정에서 사용했던 `init=/bin/sh` 옵션도 GRUB 편집 기능을 이용한 것이다.

또한 여러 운영체제를 하나의 컴퓨터에서 설치한 경우 멀티 부팅(Multi Boot)을 지원한다. 예를 들어, Linux와 Windows가 함께 설치되어 있다면 부팅 시 원하는 운영체제를 선택할 수 있다.

GRUB는 대화형 인터페이스를 제공하므로 운영체제에 문제가 발생하더라도 커널 파일 위치와 부팅 옵션만 알고 있다면 직접 수정하여 시스템을 복구할 수 있다.

GRUB2의 장점

GRUB2는 기존 GRUB보다 기능이 크게 향상된 버전이다. 셸 스크립트를 지원하기 때문에 조건문과 함수를 사용할 수 있으며, 필요한 기능을 모듈 형태로 동적으로 로드할 수 있다. 또한 그래픽 기반의 부팅 메뉴를 제공하여 사용자 편의성이 향상되었고, ISO 파일을 직접 부팅할 수 있는 기능도 지원한다.

설정 파일 구조 역시 기존보다 복잡해졌지만, 그만큼 다양한 기능을 포함할 수 있도록 개선되었다.

GRUB2 설정 파일 구조

GRUB2의 실제 설정 파일은 다음 위치에 존재한다.

```
/boot/grub2/grub.cfg
```

이 파일에는 부팅 메뉴, 커널 정보, 부팅 옵션 등이 모두 저장되어 있다.
하지만 중요한 점은 grub.cfg 파일을 직접 수정하면 안 된다는 것이다.

grub.cfg는 여러 설정 파일을 기반으로 자동 생성되는 파일이므로 직접 수정해도 다음 설정 갱신 시 덮어써질 수 있다.

실제 설정은 다음 위치에서 관리한다.

```
/etc/default/grub
```

```
/etc/grub.d
```

구조를 보면 다음과 같다.

```
/etc/default/grub
+
/etc/grub.d/*
↓
grub2-mkconfig
↓
/boot/grub2/grub.cfg 생성
```

즉, 관리자는 /etc/default/grub 또는 /etc/grub.d/ 내부 파일을 수정한 뒤 새로운 설정 파일을 생성해야 한다.

GRUB2 설정 적용

설정을 변경한 후에는 다음 명령으로 새로운 GRUB 설정 파일을 생성한다.

```
grub2-mkconfig -o /boot/grub2/grub.cfg
```

이 명령은 현재 설정 내용을 읽어 grub.cfg를 다시 생성하는 역할을 한다.

설정 파일 수정



grub2-mkconfig 실행



grub.cfg 재생성



재부팅 후 적용

GRUB2 비밀번호 설정

GRUB 메뉴 자체를 보호하고 싶다면 비밀번호를 설정할 수 있다.

```
grub2-setpassword
```

명령을 실행하면 새로운 비밀번호를 입력하게 되며, 이후에는 GRUB 편집 모드 진입이나 특정 부팅 옵션 변경 시 비밀번호를 요구하게 된다.

이는 서버 환경에서 매우 중요한 보안 기능이다.

예를 들어, Root 비밀번호 복구 과정에서 사용한 shift + e 편집 기능은 GRUB 비밀번호가 설정되어 있지 않을 때 누구나 사용할 수 있다. 따라서 물리적으로 서버에 접근 가능한 환경에서는 GRUB 비밀 번호 설정을 통해 무단 부팅 설정 변경을 방지할 수 있다.

정리

GRUB는 Linux 시스템의 부트로더로서 운영체제를 실행하고 부팅 과정을 관리하는 핵심 프로그램이다. 사용자는 GRUB를 통해 부팅 옵션을 수정하거나 복구 모드로 진입할 수 있으며, 여러 운영체제를 선택하여 부팅할 수도 있다.

실제 설정 파일은 /boot/grub2/grub.cfg 이지만 직접 수정하지 않고 /etc/default/grub 및 /etc/grub.d를 수정한 뒤 grub2-mkconfig 명령으로 설정을 반영한다.

또한, grub2-setpassword를 이용하여 GRUB 자체에 비밀번호를 설정함으로써 부팅 단계에서의 보안을 강화할 수 있다.

6. 커널 모듈과 커널 컴파일

Linux 커널은 운영체제의 핵심 부분으로 CPU, 메모리, 디스크, 네트워크와 같은 하드웨어를 제어하고 응용프로그램과 하드웨어 사이를 연결하는 역할을 수행한다.

하지만 현대 운영체제는 지원해야 하는 하드웨어와 기능이 매우 많기 때문에 모든 기능을 항상 커널 내부에 포함시키면 커널 크기가 지나치게 커지고 메모리 사용량도 증가하게 된다.

이 문제를 해결하기 위해 Linux는 모듈(Module)이라는 개념을 사용한다.

모듈은 커널의 기능 중 자주 사용되지 않거나 특정 환경에서만 필요한 기능을 별도의 코드 형태로 분리해 둔 것이다. 필요할 때만 메모리에 적재하여 사용하고, 사용하지 않을 때는 제거할 수 있다.

예를 들어, 특정 네트워크 카드 드라이버나 USB 장치 드라이버는 항상 필요한 것이 아니다. 따라서 이러한 기능들은 모듈 형태로 제공되고 장치가 연결될 때만 로드된다.

Linux Kernel

├─ CPU 관리

├─ 메모리 관리

├─ 프로세스 관리

└─ 모듈(Module)

 ├─ 네트워크 드라이버

 ├─ USB 드라이버

 └─ 파일시스템 드라이버

이러한 구조 덕분에 커널은 필요한 기능만 동적으로 추가하여 사용할 수 있으며, 메모리를 효율적으로 사용할 수 있다.

커널 컴파일은 커널 소스 코드를 이용하여 새로운 커널을 생성하는 과정이다.

일반 사용자는 배포판에서 제공하는 커널을 사용하지만, 특정 기능을 추가하거나 불필요한 기능을 제거하거나 최신 기능을 사용하기 위해 직접 커널을 컴파일하기도 한다.

중요한 점은 커널을 업그레이드한다고 해서 기존 커널이 삭제되는 것이 아니라는 것이다.

Linux는 여러 버전의 커널을 동시에 보관할 수 있으며, 문제가 발생하면 이전 커널로 부팅할 수 있다. 이러한 이유로 커널 업데이트는 비교적 안전하게 수행할 수 있다.

현재 사용 중인 커널 버전은 다음 명령으로 확인할 수 있다.

```
uname -r
```

커널 소스는 일반적으로 Linux 커널 공식 사이트인 kernel.org에서 다운로드 한다.

관례적으로 커널 소스는 /usr/src 디렉터리에 저장한다.

```
cd /usr/src
```

다운로드한 커널 소스를 압축 해제한다.

```
tar xvfj linux-6.16.6.tar.xz
```

압축을 해제하면 커널 소스 디렉터리가 생성된다.

새로운 커널을 설정하기 전에 기존 설정 파일이나 컴파일 결과를 초기화하기 위해 다음 명령을 사용한다.

```
make mrproper
```

이 명령은 이전 빌드 결과와 설정 정보를 제거하여 깨끗한 상태를 만든다.

이후 커널 환경 설정을 진행한다.

```
make xconfig
```

xconfig는 그래픽 환경에서 커널 기능을 선택할 수 있는 설정 도구이다.

여기서 드라이버, 파일 시스템, 네트워크 기능, 보안 기능 등을 선택하거나 제거할 수 있다.

설정 내용은 .config 파일에 저장된다. 필요에 따라 직접 수정할 수도 있으며, 기존 컴파일 결과를 제거하려면 다음 명령을 사용한다.

```
make clean
```

이 과정은 CPU 성능에 따라 수십 분에서 수 시간까지 걸릴 수 있다.

컴파일이 완료되면 모듈을 설치한다.

```
make modules_install
```

이 명령은 생성된 모듈들을 /lib/modules 아래에 설치한다.

이후 커널 자체를 시스템에 설치한다.

```
make install
```

설치가 완료되면 새로운 커널 파일과 초기 RAM 디스크(initramfs)가 /boot 디렉터리에 생성된다.

```
ls -l /boot
```

를 실행하면 기존 커널과 새로 생성된 커널 파일이 함께 존재하는 것을 확인할 수 있다.

```
/boot
├── vmlinuz-기존커널
├── initramfs-기존커널
├── vmlinuz-신규커널
└── initramfs-신규커널
```

마지막으로 GRUB 부트로더가 새로운 커널을 인식해야 한다.

현재 등록된 부팅 항목은 다음과 같이 확인할 수 있다.

```
cat /etc/grub2/grub.cfg
```

재부팅 후 GRUB 메뉴에서 새 커널을 선택하여 부팅할 수 있으며, 문제가 발생하면 기존 커널로 다시 부팅할 수도 있다.

정리하면, 커널 모듈은 자주 사용되지 않는 기능을 필요할 때만 로드하기 위해 커널에서 분리한 코드이며, 커널 컴파일은 커널 소스 코드를 직접 빌드하여 새로운 커널을 생성하는 과정이다.

Linux는 여러 커널 버전을 동시에 유지할 수 있으므로 새로운 커널 설치 후에도 기존 커널을 이용한 복구가 가능하며, 이러한 구조 덕분에 안정성과 확장성을 모두 확보할 수 있다.

제4장. Linux 디스크

1. 디스크 추가

1-1. 디스크 추가 장착과 파일 시스템 구성

Linux에서 새로운 하드디스크를 장착했다고 해서 바로 사용할 수 있는 것은 아니다. 운영체제가 인식한 디스크에 파티션을 생성하고, 파일 시스템을 만든 뒤, 특정 디렉터리에 마운트해야 비로소 저장 공간으로 사용할 수 있다. 또한 시스템 재부팅 후에도 자동으로 연결되도록 /etc/fstab 파일에 등록해야 한다.

새로운 디스크를 장착하면 Linux는 이를 장치 파일 형태로 인식한다.

예를 들어, SATA 포트에 새로운 디스크를 연결하면 다음과 같이 인식될 수 있다.

기존 디스크

/dev/sda

추가 디스크

/dev/sdb

이때 /dev/sdb는 디스크 전체를 의미하며 아직 파티션이 존재하지 않는 상태이다.

먼저 fdisk 명령을 사용하여 파티션을 생성한다.

```
fdisk /dev/sdb
```

안으로 들어오게 되는데 이때

```
n
```

을 입력하게 되면 설정을 하게 된다.

fdisk 내부에서 새로운 파티션을 생성하면 다음과 같은 구조가 된다.

하드디스크

/dev/sdb

↓

파티션 생성

↓

/dev/sdb1

여기서 /dev/sdb1은 첫 번째 파티션을 의미한다.

하지만 파티션만 생성했다고 바로 사용할 수 있는 것은 아니다.
운영체제가 데이터를 저장하려면 파일 시스템(File System)이 필요하다.
따라서 mkfs 명령을 사용하여 파일 시스템을 생성한다.

```
mkfs.ext4 /dev/sdb1
```

이 명령은 /dev/sdb1 파티션을 ext4 파일 시스템으로 초기화한다.

```
/dev/sdb1
  ↓
mkfs.ext4
  ↓
EXT4 파일 시스템 생성
```

이제 저장 공간으로 사용할 준비가 되었지만 아직 운영체제의 디렉터리 구조와 연결되어 있지 않다.

Linux에서는 저장 장치를 특정 디렉터리에 연결하여 사용하는 데 이를 마운트(mount)라고 한다.

먼저 마운트할 디렉터를 생성한다.

```
mkdir /mydata
```

그 후 다음과 같이 마운트한다.

```
mount /dev/sdb1 /mydata
```

그러면 /dev/sdb1에 저장된 데이터가 /mydata 디렉터를 통해 접근 가능해진다.

구조를 보면 다음과 같다.

```
하드디스크
/dev/sdb1
  ↓
Mount
  ↓
/mydata
```

사용자는 실제 디스크 장치를 직접 사용하는 것이 아니라 /mydata 디렉터리를 통해 접근하게 된다.

다만 현재 상태는 임시 마운트이다. 시스템을 재부팅하면 마운트 정보가 사라지므로 다시 연결해야 한다. 이를 해결하기 위해 부팅 시 자동으로 마운트되도록 /etc/fstab 파일에 등록한다.

예를 들어, 다음과 같은 내용을 추가할 수 있다.

```
/dev/sdb1 /mydata ext4 defaults 0 0
```

/dev/sdb1을 보통은 UUID로 설정한다.

UUID를 새로 생성하려면

```
blkid /dev/sdb1
```

전체 목록을 확인하고 싶다면

```
blkid
```

의미는 다음과 같다.

```
/dev/sdb1 → 사용할 파티션
/mydata   → 마운트 위치
ext4      → 파일 시스템 종류
defaults  → 기본 마운트 옵션
0 0       → dump, fsck 설정
```

설정을 저장한 후에는 다음 명령으로 문법 오류를 확인할 수 있다.

```
mount -a
```

오류가 없다면 설정이 정상적으로 적용된 것이다.

이후 시스템을 재부팅하면

```
reboot
```

부팅 과정에서 /etc/fstab 파일이 읽혀지고 자동으로 /dev/sdb1 이 /mydata에 마운트된다.

전체 과정은 다음과 같이 정리할 수 있다.

```
물리적 디스크 장착
↓
/dev/sdb 인식
↓
fdisk /dev/sdb
↓
/dev/sdb1 생성
↓
mkfs.ext4 /dev/sdb1
↓
파일 시스템 생성
↓
mount /dev/sdb1 /mydata
↓
사용 가능
↓
/etc/fstab 등록
↓
재부팅 후 자동 마운트
```

정리하면 Linux에서 새로운 디스크를 사용하기 위해서는 디스크 인식 → 파티션 생성(fdisk) → 파일 시스템 생성(mkfs) → 마운트(mount) → 자동 마운트 설정(/etc/fstab)의 과정을 거쳐야 한다.

/etc/fstab은 시스템 부팅 시 자동으로 읽혀 저장장치를 연결하는 중요한 설정 파일이며, 서버 환경에서는 추가 디스크를 영구적으로 사용하기 위해 반드시 이해해야 하는 핵심 구성 요소이다.

참고로 디스크 관점에서 보고 싶다면

```
lsblk
```

또는

```
findmnt
```

로 확인하자.

ext4

주로 리눅스 운영체제에서 사용되는 표준 저널링 파일 시스템이다. 이전 버전인 ext3의 향상된 버전으로, 대용량 파일 및 볼륨을 효율적으로 지원하며, 더 빠른 읽기/쓰기 성능과 뛰어난 안정성을 제공한다.

저널링(Journaling)

시스템이 비정상적으로 종료되거나 전원이 꺼져도(크래시 발생), 저널을 활용해 데이터를 빠르고 안전하게 복구하여 데이터 손실을 방지한다.

2. RAID

2-1. RAID 정의 및 개념

RAID 정의

- RAID(Redundant Array of Inexpensive Disks)는 여러 개의 디스크를 하나의 디스크처럼 사용함
- 비용 절감 + 신뢰성 향상 + 성능 향상의 효과를 냄

하드웨어 RAID

- 하드웨어 제조업체에서 여러 개의 하드디스크를 가지고 장비를 만들어서 그 자체를 공급
- 좀 더 안정적이지만 상당한 고가

소프트웨어 RAID

- 고가의 하드웨어 RAID의 대안
- 운영체제에서 지원하는 방식
- 저렴한 비용으로 좀 더 안전한 데이터의 저장 가능
- 교재에서 진행하는 내용은 소프트웨어 RAID

2-2. RAID 방식 비교

RAID(Redundant Array of Independent Disk)는 여러 개의 하드디스크를 하나의 논리적인 저장 장치처럼 사용하는 기술이다.

RAID를 사용하는 목적은 크게 성능 향상, 데이터 보호, 저장 공간 효율성 확보로 나눌 수 있다. RAID 방식에 따라 어떤 것은 속도를 중시하고, 어떤 것은 안정성을 중시하며, 어떤 것은 두 가지를 모두 고려한다.

단순 볼륨(Simple Volume)

단순 볼륨은 RAID를 사용하지 않고 하나의 하드디스크만 사용하는 가장 기본적인 방식이다.

디스크 1 (1TB)



사용 가능 공간 1TB

구성이 단순하고 비용이 가장 적게 들지만, 디스크가 고장 나면 데이터도 함께 손실된다. 따라서 별도의 장애 대응 기능은 제공하지 않는다.

Linear RAID

Linear RAID는 여러 개의 디스크를 하나의 큰 디스크처럼 연결하여 사용하는 방식이다.

디스크1 (1TB)



디스크2 (1TB)



총 2TB

데이터는 첫 번째 디스크부터 순서대로 저장된다.

파일 A → 디스크1 저장

파일 B → 디스크1 저장

디스크1 가득 참



파일 C → 디스크2 저장

모든 디스크 용량을 그대로 사용할 수 있으므로 공간 효율은 100%이다.

또한 디스크를 추가하여 용량을 확장하기 쉽다. 그러나 어느 하나의 디스크라도 고장나면 전체 데이터에 문제가 발생할 수 있으므로 안정성은 낮다.

RAID 0 (Striping)

RAID 0은 데이터를 여러 디스크에 동시에 분산 저장하는 방식이다.

이를 스트라이핑(Striping)이라고 한다.

데이터
A B C D E F

디스크1 : A C E
디스크2 : B D F

여러 디스크가 동시에 읽기, 쓰기를 수행하므로 RAID 방식 중 가장 높은 성능을 제공한다.

디스크 2개 × 1TB
↓
총 사용 가능 공간 2TB

공간 효율 역시 100%이다. 그러나 어느 하나의 디스크라도 고장 나면 전체 디스크가 손실된다.

디스크1 장애
↓
A,C,E 손실
↓
전체 데이터 복구 불가

따라서 속도가 중요하지만 데이터 손실 위험을 감수할 수 있는 환경에 적합하다.

RAID 1 (Mirroring)

RAID 1은 동일한 데이터를 여러 디스크에 복제하여 저장하는 방식이다.

이를 미러링(Mirroring)이라고 한다.

데이터
↓
디스크1 : A B C D
디스크2 : A B C D

모든 데이터가 두 개의 디스크에 동시에 기록되므로 한쪽 디스크가 고장 나더라도 다른 디스크를 통해 계속 사용할 수 있다.

디스크 2개 × 1TB
↓
사용 가능 공간 1TB

공간 효율은 50%로 낮지만 안정성은 매우 높다.

디스크1 장애
↓
디스크2에 동일 데이터 존재
↓
서비스 지속 가능

중요한 데이터 저장, 서버 운영, 데이터 보호가 중요한 환경에서 많이 사용된다.

RAID 5 (Striping + Parity)

RAID 5는 RAID 0의 성능과 RAID 1의 안정성을 절충한 방식이다. 최소 3개의 디스크가 필요하다.

데이터는 분산 저장되며, 동시에 패리티(Parity) 정보도 함께 저장된다.

디스크1 : 데이터
디스크2 : 데이터
디스크3 : 패리티

패리티는 특정 디스크에만 저장되지 않고 각 디스크에 번갈아 분산 저장된다.

예를 들어, 디스크 하나가 고장 나더라도 나머지 데이터와 패리티 정보를 이용해 복구할 수 있다.

디스크1 장애



디스크2 + 디스크3 정보 활용



데이터 복원 가능

용량 계산은 다음과 같다.

디스크 3개 × 1TB



사용 가능 공간 2TB

$(N - 1)$

즉, 디스크 한 개 분량은 패리티 저장에 사용된다.

RAID 5는 공간 효율과 안정성의 균형이 좋아 기업 서버에서 오랫동안 많이 사용된 RAID 방식이다. 다만 디스크 2개가 동시에 고장나면 복구할 수 없다.

RAID 6 (Dual Parity)

RAID 6은 RAID 5를 개선한 방식으로 패리티를 두 개 저장한다.

디스크1 : 데이터

디스크2 : 데이터

디스크3 : 패리티1

디스크4 : 패리티2

RAID 5는 디스크 1개 장애까지만 복구 가능하지만 RAID 6은 디스크 2개가 동시에 고장나더라도 복구가 가능하다.

디스크2개 장애



패리티 정보 활용



복구 가능

최소 4개의 디스크가 필요하며 사용 가능한 공간은 다음과 같다.

디스크 4개 × 1TB
↓
사용 가능 공간 2TB

(N - 2)

안정성은 RAID 5보다 높지만 패리티 계산이 더 많기 때문에 성능은 약간 감소한다.

RAID 10(RAID 1 + RAID 0)

RAID 10은 RAID 1의 미러링과 RAID 0의 스트라이핑을 결합한 방식이다.

먼저 RAID 1로 데이터를 복제하고, 그 위에서 RAID 0으로 분산 저장한다.

디스크1 ─┐
 └─ 미러링
디스크2 ─┐

디스크3 ─┐
 └─ 미러링
디스크4 ─┐

↓
두 그룹을 RAID0으로 구성

장점은 매우 명확하다.

- RAID 0 수준의 높은 성능
- RAID 1 수준의 높은 안정성
- 장애 발생 시 빠른 복구

반면 공간 효율은 RAID 1과 동일하게 50% 수준이다.

디스크 4개 × 1TB
↓
사용 가능 공간 2TB

비용은 높지만 성능과 안정성을 동시에 확보할 수 있기 때문에 데이터베이스 서버, 가상화 서버, 고성능 스토리지 환경에서 많이 사용된다.

2-3. Software RAID 생성 (Linear RAID 구성)

Linux에서는 mdadm 명령을 사용하여 여러 개의 물리 디스크를 하나의 RAID 장치로 구성할 수 있다. 이번 실습에서는 /dev/sdb1과 /dev/sdc1 두 개의 파티션을 이용하여 Linear RAID를 생성하였다.

Linear RAID는 여러 디스크를 하나의 큰 디스크처럼 연결하여 사용하는 방식이다. 데이터는 첫 번째 디스크부터 순차적으로 저장되며, 공간 효율은 100%이지만 장애 허용 기능은 제공하지 않는다.

RAID 구성 전 상태 확인

먼저 새로 추가한 디스크가 정상적으로 인식되었는지 확인한다.

```
ls -l /dev/sd*
```

예를 들어, 다음과 같이 확인될 수 있다.

```
/dev/sda  
/dev/sdb  
/dev/sdc
```

여기서

- /dev/sda: 운영체제가 설치된 디스크
- /dev/sdb: RAID용 디스크 1
- /dev/sdc: RAID용 디스크 2

를 의미한다.

파티션 생성

RAID를 구성하기 위해서는 먼저 각 디스크에 파티션을 생성해야 한다.

```
ls -l /dev/sd*
```

```
/dev/sdb1  
/dev/sdc1
```

구조는 다음과 같다.

디스크 1

/dev/sdb



/dev/sdb1

디스크 2

/dev/sdc



/dev/sdc1

Linear RAID 생성

이제 mdadm 명령으로 RAID 장치를 생성한다.

```
mdadm --create /dev/md9 W
```

```
--level=linear W
```

```
--raid-devices=2 W
```

```
/dev/sdb1 /dev/sdc1
```

의미는 다음과 같다.

/dev/md9 → 생성될 RAID 장치

--level=linear → Linear RAID 방식

--raid-devices=2 → 디스크 2개 사용

/dev/sdb1

/dev/sdc1 → RAID 구성 대상

생성 후 구조는 다음과 같다.

/dev/sdb1

|

|————▶ /dev/md9

|

/dev/sdc1

운영체제는 이제 /dev/sdb1, /dev/sdc1 을 따로 보지 않고 /dev/md9 라는 하나의 디스크처럼 인식한다.

파일 시스템 생성

RAID 장치는 아직 저장 공간으로 사용할 수 없다.

파일 시스템을 생성해야 한다.

```
mkfs.ext4 /dev/md9
```

/dev/md9



EXT4 파일 시스템 생성

마운트

RAID 장치를 사용할 디렉터리를 생성한다.

```
mkdir /raidLinear
```

그리고 RAID 장치를 연결한다.

```
mount /dev/md9 /raidLinear
```

구조는 다음과 같다.

/dev/md9



Mount



/raidLinear

사용자는 /raidLinear 디렉터리를 통해 RAID 저장 공간을 사용할 수 있다.

용량 확인

정상적으로 마운트되었는지 확인한다.

```
df -h
```

결과를 보면 /raidLinear에 RAID 용량이 표시된다.

예를 들어,

```
sdb = 1TB
```

```
sdc = 1TB
```

이라면

```
/raidLinear = 약 2TB
```

로 표시된다.

Linear RAID는 공간 효율이 100%이기 때문이다.

RAID 상태 확인

현재 RAID 상태는 다음 명령으로 확인할 수 있다.

```
mdadm --detail /dev/md9
```

출력 결과에는

- RAID 종류
- 사용 중인 디스크
- RAID 상태
- 동기화 여부

등이 표시된다.

현재 시스템에 존재하는 RAID 정보를 확인하려면 다음 명령을 사용한다.

```
mdadm --detail --scan
```


RAID 설정 저장

RAID는 재부팅 후에도 자동으로 인식되어야 한다.

현재 RAID 정보를 설정 파일로 저장한다.

```
mdadm --detail --scan > /etc/mdadm.conf
```

예를 들어, 다음과 같은 정보가 저장된다.

```
ARRAY /dev/md9 level=linear num-devices=2
```

자동 마운트 설정

재부팅 후 자동으로 마운트되도록 /etc/fstab에 등록한다.

```
gnome-text-editor /etc/fstab
```

다음 내용을 추가한다.

```
/dev/md9 /raidLinear ext4 defaults 0 0
```

2-4. Software RAID 재구성과 장애 복구

Software RAID를 운영하다 보면 RAID 구성을 해제하거나 디스크를 교체해야 하는 상황이 발생할 수 있다. 이때 기존 RAID 정보를 제거하고 새로운 디스크를 이용하여 RAID를 다시 구성해야 한다.

이번 실습에서는 기존 Linear RAID(/dev/md9)를 중지한 후 새로운 RAID 0을 구성하는 과정을 수행하였다.

기존 RAID 중지 및 설정 제거

RAID 장치는 일반 디스크와 달리 mdadm이 관리한다.

현재 동작 중인 RAID를 확인할 수 있으며 필요에 따라 중지할 수 있다.

```
mdadm --run /dev/md9  
mdadm --stop /dev/md9
```

--run은 RAID를 활성화하는 명령이고, --stop은 RAID를 중지하는 명령이다.

```
/dev/md9  
↓  
STOP  
↓  
비활성화
```

또한 RAID 장치가 /etc/fstab에 등록되어 있다면 부팅 시 자동 마운트가 시도된다.

```
nano /etc/fstab
```

기존에 추가했던 내용을 삭제하거나 주석 처리한다.

```
# /dev/md9 /raidLinear ext4 defaults 0 0
```

이 작업을 하지 않으면 재부팅 시 존재하지 않는 RAID 장치를 마운트하려고 하여 부팅 오류가 발생할 수 있다.

새로운 디스크 장착 확인

실제 서버에서는 고장 난 디스크를 제거하고 새로운 디스크를 장착한다.

Linux에서 디스크가 정상적으로 인식되었는지 확인한다.

```
lsblk -S
```

또는

```
ls -l /dev/sd*
```

예를 들어, 다음과 같이 표시될 수 있다.

/dev/sda : 운영체제 디스크

/dev/sdb : RAID 구성용 디스크

/dev/sdc : 새로 추가한 디스크

기존 RAID 메타데이터 제거

디스크에는 이전 RAID 구성 정보가 남아 있을 수 있다.

이를 제거하지 않으면 새로운 RAID 생성 시 오류가 발생할 수 있다.

먼저 기존 RAID를 중지한다.

```
mdadm --stop /dev/md9
```

그 후 RAID 메타데이터를 삭제한다.

```
mdadm --zero-superblock /dev/sdb1
```

```
mdadm --zero-superblock /dev/sdc1
```

구조적으로 보면 다음과 같다.

기존 RAID 정보

↓

Superblock 삭제

↓

새 디스크 상태로 초기화

현재 RAID 정보가 남아 있는지 확인하려면

```
mdadm --examine /dev/sdc1
```

명령을 사용할 수 있다.

RAID 0 생성

준비가 완료되면 RAID 0을 생성한다.

```
mdadm --create /dev/md0 W  
--level=0 W  
--raid-devices=2 W  
/dev/sdb1 /dev/sdc1
```

옵션 의미는 다음과 같다.

```
/dev/md0      → 생성될 RAID 장치  
--level=0     → RAID 0 (Striping)  
--raid-devices=2 → 디스크 2개 사용
```

생성 후 구조는 다음과 같다.

```
/dev/sdb1  
|  
|————▶ /dev/md0  
|  
/dev/sdc1
```

RAID 0은 데이터를 두 디스크에 분산 저장하므로 성능이 향상된다.

데이터
A B C D E F

디스크1
A C E

디스크2
B D F

파일 시스템 생성

RAID 장치는 생성 직후에는 사용할 수 없다.

파일 시스템을 생성해야 한다.

```
mkfs.ext4 /dev/md0
```

마운트

마운트 디렉터리를 생성한다.

```
mkdir /raid0
```

RAID 장치를 연결한다.

```
mount /dev/md0 /raid0
```

구조는 다음과 같다.

```
/dev/md0
  ↓
Mount
  ↓
/raid0
```

정리하면 RAID 장치에 문제가 발생하거나 새로운 RAID 구성을 만들고자 할 때는 기존 RAID 중지 → RAID 메타데이터 삭제 → 새로운 RAID 생성 → 파일 시스템 생성 → 마운트 순서로 작업한다.

특히, `mdadm --zero-superblock` 은 디스크에 남아 있는 이전 RAID 정보를 제거하는 매우 중요한 명령이며, 이를 수행하지 않으면 새로운 RAID 구성이 정상적으로 생성되지 않을 수 있다.

2-5. RAID 1 디스크 장애 시 복구 및 디스크 추가

RAID 1(Mirroring)은 동일한 데이터를 두 개 이상의 디스크에 동시에 저장하는 방식이다. 따라서 디스크 하나가 고장 나더라도 나머지 디스크에 동일한 데이터가 남아 있으므로 시스템을 계속 사용할 수 있다. RAID 0과 달리 디스크 장애가 발생해도 데이터가 유지된다는 것이 가장 큰 장점이다.

예를 들어, RAID 1 구성이 다음과 같다고 가정한다.

```
/dev/md1
├── /dev/sdb1
└── /dev/sdc1
```

이 상태에서 /dev/sdc1 디스크가 고장 나면 RAID는 Degraded(성능 저하) 상태로 동작하게 된다.

현재 상태는 다음과 같이 확인할 수 있다.

```
cat /proc/mdstat
mdadm --detail /dev/md1
```

출력 결과에서는 한 개의 디스크가 제거되었거나 Failed 상태로 표시되는 것을 확인할 수 있다.

디스크가 고장 난 경우에는 새로운 디스크를 장착한 뒤 기존 RAID 배열에 다시 추가해야 한다. 먼저 새 디스크를 시스템이 인식했는지 확인한다.

```
lsblk -S
```

새 디스크가 /dev/sdc로 인식되었다면 기존 RAID 구성과 동일한 크기의 파티션을 생성한다.

```
fdisk /dev/sdc
```

파티션 생성 후에는 RAID 메타데이터가 남아 있는 경우를 대비해 초기화한다.

```
mdadm --zero-superblock /dev/sdc1
```

이후 RAID 배열에 새 디스크를 추가한다.

```
mdadm /dev/md1 --add /dev/sdc1
```

또는

```
mdadm --manage /dev/md1 --add /dev/sdc1
```

명령을 사용할 수 있다.

디스크를 추가하면 RAID는 자동으로 Rebuild(재동기화) 작업을 시작한다.

이 과정에서는 살아 있는 디스크의 데이터를 새 디스크에 복사하여 동일한 상태로 만든다.

진행 상황은 다음 명령으로 확인할 수 있다.

```
cat /proc/mdstat
```

결론적으로 RAID 1에서는 디스크 하나가 고장나더라도 데이터가 유지된다.

새로운 디스크를 장착한 후 파티션을 생성하고 mdadm --add 명령으로 기존 RAID 배열에 추가하면 자동으로 데이터를 복제하여 원래 상태를 복구할 수 있다.

2-6. LVM(Logical Volume Manager)

LVM이란

LVM(Logical Volume Manager)은 여러 개의 물리 디스크 또는 파티션을 하나의 저장 공간으로 통합한 뒤, 이를 다시 원하는 크기로 나누어 사용할 수 있도록 해주는 디스크 관리 기술이다.

일반적인 파티션 방식에서는 디스크 크기가 고정되기 때문에 나중에 용량을 변경하기 어렵다. 반면, LVM은 물리 디스크와 파일 시스템 사이에 논리적인 계층을 추가하여 저장 공간을 유연하게 관리할 수 있도록 한다.

예를 들어, 2TB 디스크 2개가 있다고 가정하면 일반적인 방식에서는 각각 독립적으로 사용해야 한다.

디스크1 : 2TB

디스크2 : 2TB

하지만 LVM을 사용하면 두 디스크를 하나의 저장 공간으로 합칠 수 있다.

2TB + 2TB

↓

4TB 저장 공간

그리고 이를 다시 원하는 크기로 나누어 사용할 수 있다.

4TB

├── 1TB (LV1)

└── 3TB (LV2)

즉, 물리적인 디스크 구조와 사용자가 사용하는 저장 공간을 분리하여 관리할 수 있는 것이 LVM의 핵심 개념이다.

LVM의 구조

LVM은 크게 세 단계로 구성된다.

```
Physical Volume(PV)
  ↓
Volume Group(VG)
  ↓
Logical Volume(LV)
  ↓
File System
```

Physical Volume(PV)

PV은 LVM이 사용할 실제 디스크 또는 파티션이다.

예를 들어, 다음과 같은 파티션이 있다면

```
/dev/sda1
/dev/sdb1
```

이 두 파티션을 LVM의 물리 볼륨으로 등록할 수 있다.

```
pvcreate /dev/sdb1
pvcreate /dev/sdc1
```

Volume Group(VG)

Volume Group은 여러 개의 Physical Volume을 하나로 묶어 만든 저장 공간이다.

```
/dev/sdb1
/dev/sdc1
  ↓
myVG
```

생성 명령은 다음과 같다.

```
vgcreate myVG /dev/sdb1 /dev/sdc1
```

현재 생성된 VG 정보는 다음과 같이 확인할 수 있다.

```
vgdisplay
```

Logical Volume(LV)

Logical Volume은 Volume Group 내부에서 생성하는 논리적인 파티션이다.

사용자는 실제 디스크가 아니라 LV를 일반 파티션처럼 사용하게 된다.

예를 들어, 100GB의 VG가 있다면 다음과 같이 여러 개의 LV를 만들 수 있다.

```
myVG
├── myLV1 (10GB)
├── myLV2 (20GB)
└── myLV3 (나머지 전체)
```

LV 생성 예시는 다음과 같다.

```
lvcreate --size 10G --name myLV1 myVG
```

남은 공간 전체를 사용하려면

```
lvcreate --extents 100%FREE --name myLV3 myVG
```

를 사용한다.

파일 시스템 생성 및 마운트

LV를 생성했다고 바로 사용할 수 있는 것은 아니다.

일반 파티션과 마찬가지로 파일 시스템을 생성해야 한다.

```
mkfs.ext4 /dev/myVG/myLV1
```

그 후 마운트 디렉터리를 생성한다.

```
mkdir /lvm1
```

마운트한다.

```
mount /dev/myVG/myLV1 /lvm1
```

그러면 /lvm1 디렉터리를 통해 LVM 공간을 사용할 수 있다.

부팅 시 자동 마운트

재부팅 후에도 자동으로 마운트되도록 하려면 `/etc/fstab` 파일에 등록해야 한다.

```
gnome-text-editor /etc/fstab
```

예시:

```
/dev/myVG/myLV1 /lvm1 ext4 defaults 0 0
```

설정 저장 후 재부팅하면 자동으로 마운트된다.

LVM의 장점

LVM의 가장 큰 장점은 저장 공간을 유연하게 관리할 수 있다는 것이다.

일반 파티션은 생성 후 크기 변경이 어렵지만, LVM은 필요에 따라 Logical Volume의 크기를 늘리거나 줄일 수 있다. 또한 새로운 디스크를 추가하여 Volume Group의 크기를 확장할 수도 있다.

기존

```
/dev/sdb1 + /dev/sdc1
```

↓

```
myVG
```

새 디스크 추가

```
/dev/sdd1
```

↓

```
vgextend myVG /dev/sdd1
```

이렇게 하면 기존 데이터를 유지한 상태에서 저장 공간을 확장할 수 있다.

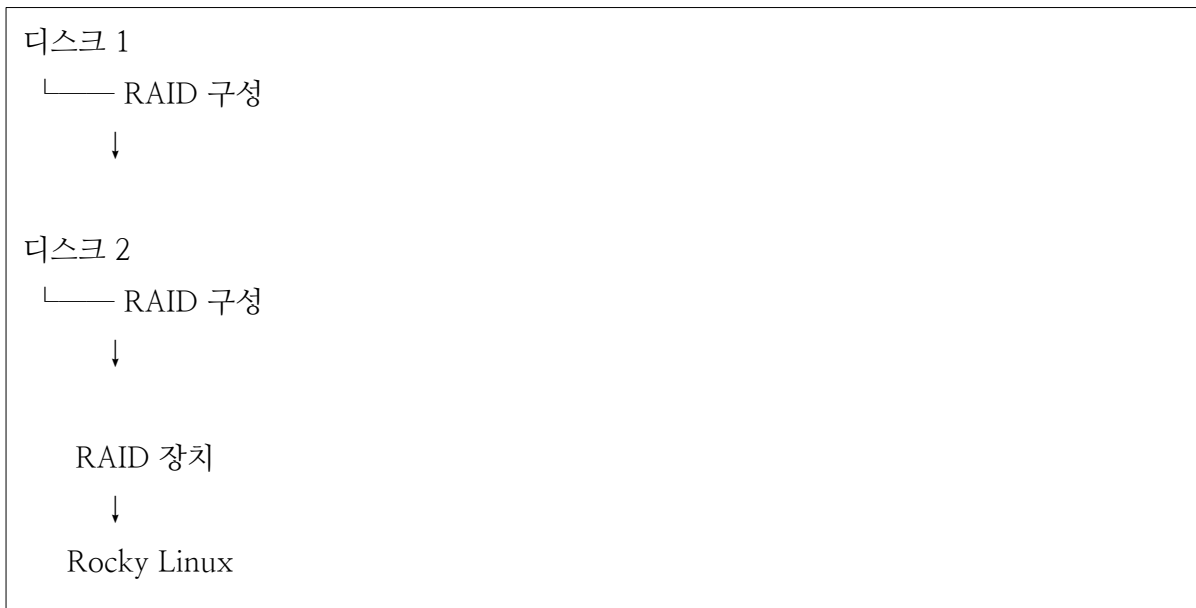
2-7. RAID를 운영체제 설치 시 적용하는 방법

지금까지 운영체제를 설치한 후 추가 디스크를 이용하여 Software RAID를 구성하였다. 그러나 RAID는 데이터 저장용 디스크뿐만 아니라 운영체제가 설치되는 디스크 자체에도 적용할 수 있다.

예를 들어, Rocky Linux 설치과정에서 디스크 파티션을 설정할 때 RAID 구성을 선택하면 운영체제가 설치되는 영역을 RAID로 구성할 수 있다.

즉, 운영체제 파일, 프로그램, 사용자 데이터 등이 모두 RAID 위에 저장되는 구조가 된다.

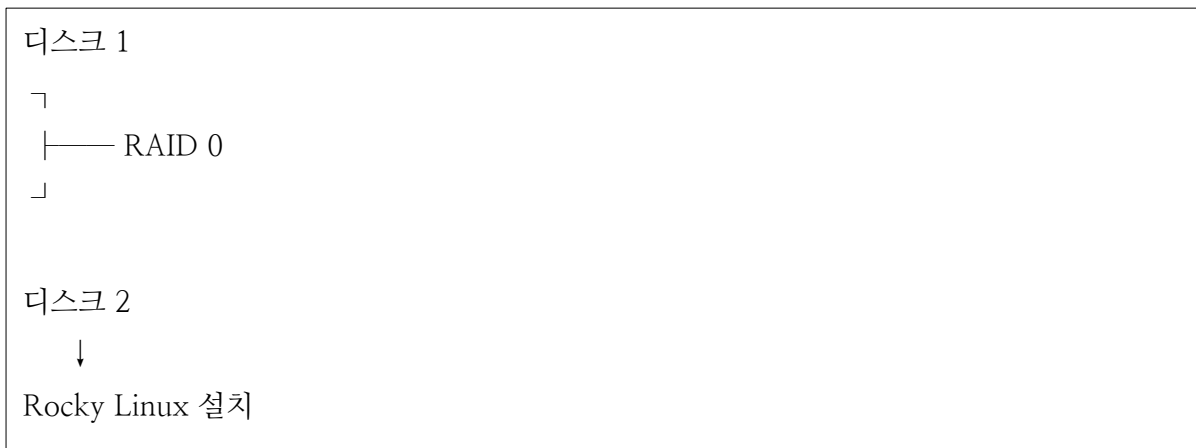
설치 과정에서 RAID를 구성하면 다음과 같은 형태가 될 수 있다.



특히, RAID 1은 서버 환경에서 자주 사용된다.

운영체제가 설치된 디스크 중 하나가 고장 나더라도 다른 디스크에 동일한 데이터가 저장되어 있으므로 시스템을 계속 부팅하고 사용할 수 있기 때문이다.

반면 RAID 0은 운영체제 설치 시 적용할 수 있다.



RAID 0은 두 디스크에 데이터를 분산 저장(Striping)하여 성능을 높일 수 있지만, 디스크 하나만 고장나도 운영체제 전체가 손상된다. 따라서 성능은 뛰어나지만 안정성은 매우 낮다.

실제로 VMware와 같은 가상환경에서도 가상머신 생성 후 운영체제를 설치할 때 RAID 구성을 선택할 수 있으며, 이 경우 운영체제 자체가 RAID 위에서 동작하게 된다.

결론적으로 RAID는 운영체제 설치 후 추가 저장공간에만 적용하는 것이 아니라, Linux 설치 과정에서 시스템 디스크 자체를 RAID로 구성할 수도 있다.

다만 운영체제용 RAID는 안정성이 중요한 경우 RAID 1을 사용하는 것이 일반적이며, RAID 0은 성능 향상 목적일 때만 제한적으로 사용된다.

2-8. 디스크 사용량 제한(Quota)

Quota란?

Quota(쿼터)는 특정 사용자가 사용할 수 있는 디스크 공간과 파일 개수를 제한하는 기능이다. 서버 환경에서는 여러 사용자가 하나의 시스템을 함께 사용하기 때문에, 특정 사용자가 과도하게 디스크를 사용하면 다른 사용자가 사용할 공간이 부족해질 수 있다.

예를 들어, 10GB 디스크를 여러 사용자가 함께 사용하는 경우, 한 사용자가 9GB를 사용해 버리면 나머지 사용자는 거의 사용할 수 있는 공간이 남지 않게 된다.

이러한 문제를 방지하기 위해 사용자별 또는 그룹별 디스크 사용량 제한 기능인 Quota를 사용한다.

특히 루트 파일시스템(/)에 직접 Quota를 적용하기보다는 별도의 파일시스템을 생성하여 사용자 데이터를 저장하는 것이 일반적이다. 운영체제가 사용하는 영역과 사용자 데이터 영역을 분리하면 시스템 성능과 안정성을 높일 수 있기 때문이다.

사용자 전용 파일시스템 생성

실습에서는 10GB 디스크를 추가한 후 사용자 홈 디렉터리 전용 공간으로 구성하였다. 먼저 새 디스크에 파티션을 생성한다.

```
fdisk /dev/sdb
```

파일시스템을 생성한다.

```
mkfs.ext4 /dev/sdb1
```

마운트할 디렉터를 생성한다.

```
mkdir /userHome
```

마운트한다.

```
mount /dev/sdb1 /userHome
```

부팅 시 자동 마운트를 위해 /etc/fstab에 등록한다.

```
nano /etc/fstab  
/dev/sdb1 /userHome ext4 defaults 0 0
```

사용자 생성

이제 사용자 홈 디렉터리를 /userHome 아래에 생성한다.

```
useradd -d /userHome/blackpink blackpink
useradd -d /userHome/mamamoo mamamoo
```

비밀번호를 설정한다.

```
passwd blackpink
passwd mamamoo
```

이제 각 사용자는 /userHome 아래 자신의 홈 디렉터를 사용하게 된다.

Quota 기능 활성화

Quota를 사용하려면 먼저 파일 시스템이 Quota 기능을 지원하도록 설정해야 한다.

/etc/fstab의 마운트 옵션에 다음 내용을 추가한다.

```
/dev/sdb1 /userHome ext4 defaults,usrquota,grpquota 0 0
```

- usrquota: 사용자별 Quota 활성화
- grpquota: 그룹별 Quota 활성화

왜 umount를 하는가?

Quota 기능을 활성화하는 과정에서 다음 명령을 수행한다.

```
umount /dev/sdb1
```

마운트를 해제하는 이유는 파일시스템의 내부 기능을 변경해야 하기 때문이다.

이후 실행하는

```
tune2fs -O quota /dev/sdb1
```

명령은 ext4 파일시스템에 Quota 기능을 추가하는 작업이다.

파일 시스템이 사용 중인 상태에서는 이러한 메타데이터 변경이 불가능하거나 위험할 수 있으므로 먼저 마운트를 해제한 후 작업을 수행한다.

즉,

사용 중인 파일시스템



umount



파일시스템 기능 수정



다시 mount

과정을 거치는 것이다.

Quota 기능 적용

Quota 기능을 활성화한다.

```
tune2fs -O quota /dev/sdb1
```

설정 변경 후 systemd 설정을 다시 읽어들인다.

```
systemctl daemon-reload
```

다시 마운트한다.

```
mount /dev/sdb1 /userHome
```

또는

```
mount -o remount /userHome
```

현재 상태를 확인한다.

```
mount | tail -3
```


Quota 데이터베이스 생성

Quota는 사용자 사용량 정보를 별도의 데이터베이스 파일에 저장한다.

먼저 기존 Quota 기능을 비활성화한다.

```
quotaoff -avug
```

Quota 데이터베이스를 생성한다.

```
quotacheck -augmn
```

실습 중 오류가 발생하면 기존 파일을 삭제할 수 있다.

```
rm -rf aquota.*
```

Quota 파일을 생성한다.

```
touch aquota.user aquota.group  
chmod 600 aquota.*
```

다시 데이터베이스를 생성한다.

```
quotacheck -augmn
```

생성 결과를 확인하려면 다음 파일이 존재한다.

```
/userHome  
├── aquota.user  
└── aquota.group
```

이 파일들이 사용자 및 그룹 Quota 정보를 저장하는 데이터베이스 역할을 한다.

Quota 활성화

Quota 기능을 활성화한다.

```
quotaon -avug
```

옵션 의미는 다음과 같다.

- a: 모든 파일시스템
- v: 상세 출력
- u: 사용자 Quota
- g: 그룹 Quota

현재 활성화 여부는 다음과 같이 확인할 수 있다.

```
mount
```

```
quotaon -p
```

사용자별 Quota 설정

사용자별 제한은 edquota 명령으로 설정한다.

예를 들어, Blackpink 사용자에게 1GB 제한을 설정하려면

```
edquota blackpink
```

를 실행한다.

그러면 편집기가 열리며 다음과 같은 항목을 수정할 수 있다.

Soft Limit	: 권장 한도
Hard Limit	: 절대 한도

예를 들어,

```
Soft Limit : 900MB  
Hard Limit : 1GB
```

로 설정하면 사용자는 최대 1GB까지만 사용할 수 있다.

정리

Quota는 사용자나 그룹이 사용할 수 있는 디스크 공간과 파일 개수를 제한하는 기능이다.

사용자



Quota 검사



허용 범위 이내



파일 생성 가능

허용 범위 초과



파일 생성 불가

서버 환경에서는 특정 사용자가 디스크 공간을 독점하는 것을 방지하고 안정적으로 저장 공간을 관리하기 위해 Quota 기능을 사용한다. 일반적으로 별도의 파일시스템을 생성한 후 `usrquota`, `grpquota` 옵션을 활성화하고, `quotacheck`, `quotaon`, `edquota` 명령을 이용하여 사용자별 사용량을 관리한다.